

Unit – 3 (CPU SCHEDULING)

Process

A process is a program at the time of execution.

Differences between Process and Program

Process	Program
Process is a dynamic object	Program is a static object
Process is sequence of instruction execution	Program is a sequence of instructions
Process loaded in to main memory	Program loaded into secondary storage devices
Time span of process is limited	Time span of program is unlimited
Process is a active entity	Program is a passive entity

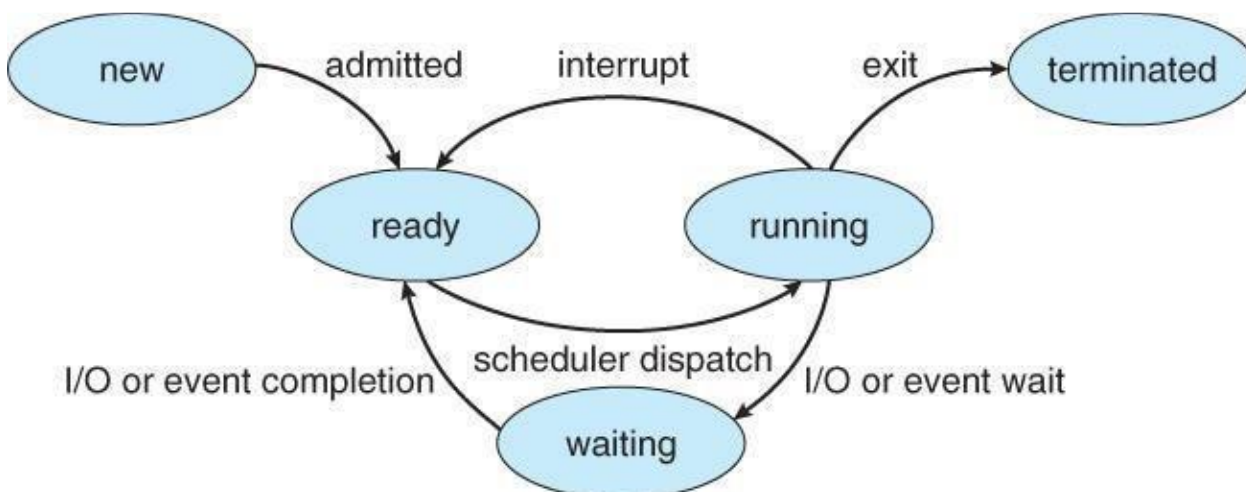
Process States

When a process executed, it changes the state, generally the state of process is determined by the current activity of the process. Each process may be in one of the following states:

1. New : The process is being created.
2. Running : The process is being executed.
3. Waiting : The process is waiting for some event to occur.
4. Ready : The process is waiting to be assigned to a processor.
5. Terminated : The Process has finished execution.

Only one process can be running in any processor at any time, But many process may be in ready and waiting states. The ready processes are loaded into a “ready queue”.

Process Transition Diagram



A) New ->Ready : OS creates process and prepares the process to be executed then OS moved the process into ready queue.

b) Ready->Running : OS selects one of the Jobs from ready Queue and move them from ready to Running.

c) Running->Terminated : When the Execution of a process has Completed, OS terminates that process from running state. Sometimes OS terminates the process for some other reasons including Time exceeded, memory unavailable, access violation, protection Error, I/O failure and soon.

d) Running->Ready : When the time slot of the processor expired (or) If the processor received any interrupt signal, the OS shifted Running -> Ready State.

e) Running -> Waiting : A process is put into the waiting state, if the process need an event occur (or) an I/O Device require.

f) Waiting->Ready : A process in the waiting state is moved to ready state when the event for which it has been Completed.

Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process as listed below in the table –

S.N.	Information & Description
1	Process State: The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	Process privileges: This is required to allow/disallow access to system resources.
3	Process ID: Unique identification for each of the process in the operating system.
4	Pointer: A pointer to parent process.
5	Program Counter: Program Counter is a pointer to the address of the next instruction to be executed for this process.

6	CPU registers: Various CPU registers where process need to be stored for execution for running state.
7	CPU Scheduling Information: Process priority and other scheduling information which is required to schedule the process.
8	Memory management information: This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	Accounting information: This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	IO status information: This includes a list of I/O devices allocated to the process.

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB –

Role of PCB:

- The most important data structure in an OS
 - ☐ contains all of the information about a process that is needed by the OS
 - ☐ blocks are read and/or modified by virtually every module in the OS
 - ☐ defines the state of the OS
- Difficulty is not access, but protection
 - ☐ a bug in a single routine could damage process control blocks, which could destroy the system's ability to manage the affected processes
 - ☐ a design change in the structure or semantics of the process control block could affect a number of modules in the OS

Process ID
State
Pointer
Priority
Program counter
CPU registers
I/O information
Accounting information
etc....

The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

Scheduling Concepts

CPU is always busy in **Multiprogramming**. Because CPU switches from one job to another job. But in **simple computers** CPU sit idle until the I/O request granted. **scheduling** is a important OS function. All resources are scheduled before use.(cpu, memory, devices.....)

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing..

Scheduling Objectives

- Maximize throughput.
- Maximize number of users receiving acceptable response times.
- Be predictable.
- Balance resource use.
- Avoid indefinite postponement.
- Enforce Priorities.

SCHEDULING QUEUES:

People live in rooms. Process are present in rooms knows as queues. There are 3types of queues.

- 1. Job queue:** when processes enter the system, they are put into a **job queue**, which consists all processes in the system. Processes in the job queue reside on mass storage and await the allocation of main memory.
- 2. Ready queue:** if a process is present in main memory and is ready to be allocated to cpu for execution, is kept in **ready queue**.

3. Device queue: if a process is present in waiting state (or) waiting for an i/o event to complete is said to be in device queue.(or) The processes waiting for a particular I/O device is called device queue.

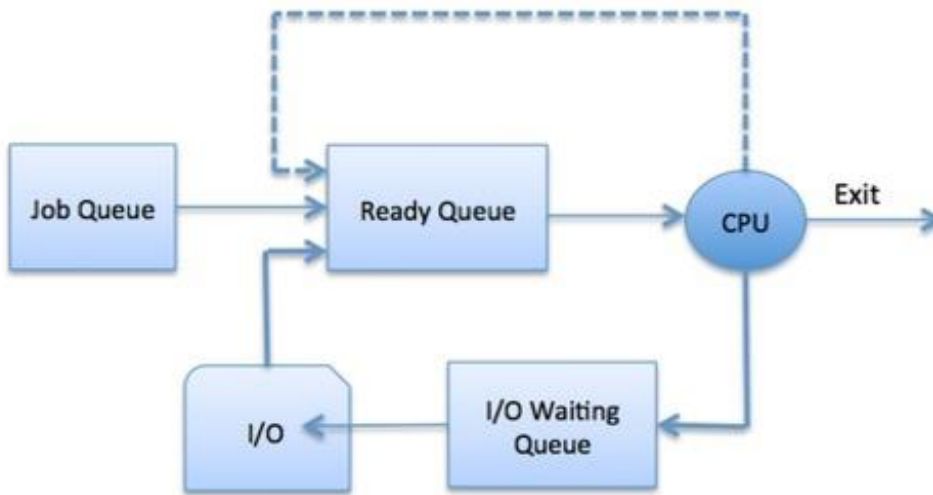


Fig.: Queuing Diagram

Schedulers :.

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Types of schedulers

1. Long term scheduler:

select the jobs from the job pool and loaded these jobs into main memory (ready queue). Long term scheduler is also called job scheduler.

2. Short term scheduler:

select the process from ready queue, and allocates it to the cpu. If a process requires an I/O device, which is not present available then process enters device queue. short term scheduler maintains ready queue, device queue. Also called as cpu scheduler.

3. Medium term scheduler: if process request an I/O device in the middle of the execution, then the process removed from the main memory and loaded into the waiting queue. When the I/O operation completed, then the job moved from waiting queue to ready queue. These two operations performed by medium term scheduler.

Comparison between Scheduler

S.N.	Long Term Scheduler	Short Term Scheduler	Medium Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Scheduler duties:

- ☐ Maintains the queue.
- ☐ Select the process from queues assign to CPU.

Some other basic terms:

Context Switch: Assume, main memory contains more than one process. If cpu is executing a process, if time expires or if a high priority process enters into main memory, then the scheduler saves information about current process in the PCB and switches to execute the another process. The concept of moving CPU by scheduler from one process to other process is known as context switch.

Dispatcher: The main job of dispatcher is switching the cpu from one process to another process. Dispatcher connects the cpu to the process selected by the short term scheduler.

Dispatcher latency: The time it takes by the dispatcher to stop one process and start another process is known as dispatcher latency. If the dispatcher latency is increasing, then the degree of multiprogramming decreases.

Arrival Time (AT) :

Arrival time is the point of time in milli seconds at which a process arrives at the ready queue to begin the execution.

Arrival Time (A.T.) = Completion Time (C.T.) - Turn Around Time (T.A.T)

Burst Time (BT) :

Burst Time refers to the time required in milli seconds by a process for its execution.

Burst Time (B.T.)= Completion Time (C.T.) - Waiting Time (W.T.)

Performance criteria:

Various criteria or characteristics that help in designing a good scheduling algorithm are:

1. **Throughput:** how many jobs are completed by the cpu with in a time period.

$\text{Throughput} = (\text{Number of processes completed}) / (\text{Time Unit})$

2. **Turnaround time :** The time interval between the submission of the process and time of the completion is turnaround time.

$\text{TAT} = \text{Waiting time in ready queue} + \text{executing time} + \text{waiting time in waiting queue for I/O.}$

Or $\text{tat} = t(\text{process completed}) - t(\text{process submitted})$

3. **Waiting time:** The time spent by the process to wait for cpu to be allocated.

4. **Response time:** Time duration between the submission and first response.

$\text{rt} = t(\text{first response}) - t(\text{submission of request})$

5. **Cpu Utilization:** CPU is costly device, it must be kept as busy as possible.

$\text{Processor Utilization} = (\text{Processor busy time}) / (\text{Processor busy time} + \text{Processor idle time})$

Eg: CPU efficiency is 90% means it is busy for 90 units, 10 units idle.

Preemptive and Non-Preemptive Scheduling

1. Preemptive Scheduling:

Preemptive scheduling is used when a process switches from running state to ready state or from waiting state to ready state. The resources (mainly CPU cycles) are allocated to the process for the limited amount of time and then is taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in ready queue till it gets next chance to execute.

Algorithms based on preemptive scheduling are: Round Robin (RR), Shortest Job First (SJF basically non preemptive) and Priority (non preemptive version), etc.

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Preemptive Scheduling

2. Non-Preemptive Scheduling:

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to waiting state. In this scheduling, once the resources (CPU cycles) is allocated to a process, the process holds the CPU till it gets terminated or it reaches a waiting state. In case of non-preemptive scheduling does not interrupt a process running CPU in middle of the execution. Instead, it waits till the process complete its CPU burst time and then it can allocate the CPU to another process.

Algorithms based on preemptive scheduling are: Shortest Remaining Time First (SRTF), Priority (preemptive version), etc.

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Non-Preemptive Scheduling

Differences Between Preemptive and Non-Preemptive Scheduling:

Comparison Chart:

Parameter	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Basic	In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state.
Interrupt	Process can be interrupted in between.	Process cannot be interrupted untill it terminates itself or its time is up.

Parameter	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Starvation	If a process having high priority frequently arrives in the ready queue, low priority process may starve.	If a process with long burst time is running CPU, then later coming process with less CPU burst time may starve.
Overhead	It has overheads of scheduling the processes.	It does not have overheads.
Flexibility	flexible	rigid
Cost	cost associated	no cost associated

Process Address Space

The process address space is the set of logical addresses that a process references in its code. For example, when 32-bit addressing is in use, addresses can range from 0 to 0x7fffffff; that is, 2^{31} possible numbers, for a total theoretical size of 2 gigabytes.

The operating system takes care of mapping the logical addresses to physical addresses at the time of memory allocation to the program. There are three types of addresses used in a program before and after memory is allocated –

S.N.	Memory Addresses & Description
1	Symbolic addresses: The addresses used in a source code. The variable names, constants, and instruction labels are the basic elements of the symbolic address space.
2	Relative addresses: At the time of compilation, a compiler converts symbolic addresses into relative addresses.
3	Physical addresses: The loader generates these addresses at the time when a program is loaded into main memory.

Virtual and physical addresses are the same in compile-time and load-time address-binding schemes. Virtual and physical addresses differ in execution-time address-binding scheme.

The set of all logical addresses generated by a program is referred to as a **logical address space**. The set of all physical addresses corresponding to these logical addresses is referred to as a **physical address space**.

The runtime mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device. MMU uses following mechanism to convert virtual address to physical address.

- The value in the base register is added to every address generated by a user process, which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.
- The user program deals with virtual addresses; it never sees the real physical addresses.

Process Identification Information

In a computer system each time a process is created, it is given the unique identification number by the operating system in order to uniquely identify a process and differentiate one process from another. This unique identification number is called the process identifier(PID). They are positive and non zero integers. PIDs can be 16 bit, 32bit or 64bit integer values depending upon the system requirement. PIDs are usually assigned to process in sequential manner. When the n maximum range of PIDs is reached, PIDs need to be repeated means next PID rolls or starts over. At any given time , no two processes with same PID can exist in the system because it is unique identification number used by OS to manage and control the processes. Apart from process identifier, OS also maintains some other identifiers information. These are user identifier (UID) and process group identifier(PGID).

Each process in a system has a associated user with it and each user in a system is given the UID. The UID information of the user specifies user access rights, allocated resources etc. This UID information of user associated with a process is stored in PCB(Process Control Block) of that process so that process access right, allocated resources are restricted according to user rights.

Process group identifier is the unique identification number to a group of process that are occupied in performing common task. All the processes in a group will have the same PGID value but each process will have its own PID value. Different process group will have their unique PGID value. Processes in the group can be added and eliminated as needed. The process group is flexible enough to grow and minimize. OS assigns PGID so as to ease inter process communication and to share information among all the processes in the group.

Threads:

A process is divide into number of light weight process, each light weight process is said to be a Thread. The Thread has a program counter (Keeps track of which instruction to execute next), registers (holds its current working variables), stack (execution History).

Thread States:

1. bornState : A thread is justcreated.
2. readystate : The thread is waiting forCPU.

3. running : System assigns the processor to the thread.
4. sleep : A sleeping thread becomes ready after the designated sleep time expires.
5. dead : The Execution of the thread finished.

Eg: Word processor.

Typing, Formatting, Spell check, saving are threads.

Differences between Process and Thread

Process	Thread
Process takes more time to create.	Thread takes less time to create.
it takes more time to complete execution & terminate.	Less time to terminate.
Execution is very slow.	Execution is very fast.
It takes more time to switch b/w two processes.	It takes less time to switch b/w two threads.
Communication b/w two processes is difficult .	Communication b/w two threads is easy.
Process can't share the same memory area.	Threads can share same memory area.
System calls are requested to communicate each other.	System calls are not required.
Process is loosely coupled.	Threads are tightly coupled.
It requires more resources to execute.	Requires few resources to execute.

Multithreading

A process is divided into number of smaller tasks each task is called a Thread. Number of Threads within a Process execute at a time is called Multithreading.

If a program is multithreaded, even when some portion of it is blocked, the whole program is not blocked. The rest of the program continues working. If multiple CPU's are available.

Multithreading gives best performance. If we have only a single thread, number of CPU's available, No performance benefits achieved.

- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

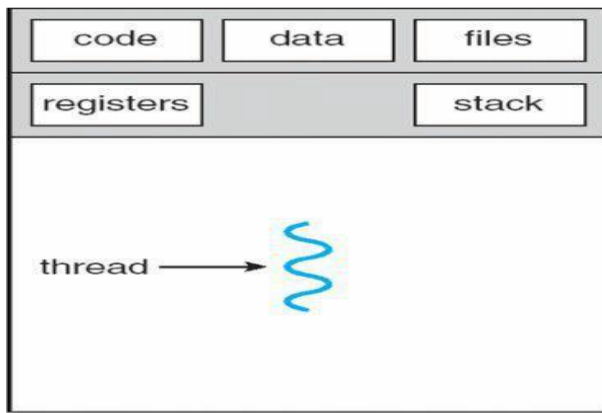
CODE- Contains instruction

DATA- holds global variable

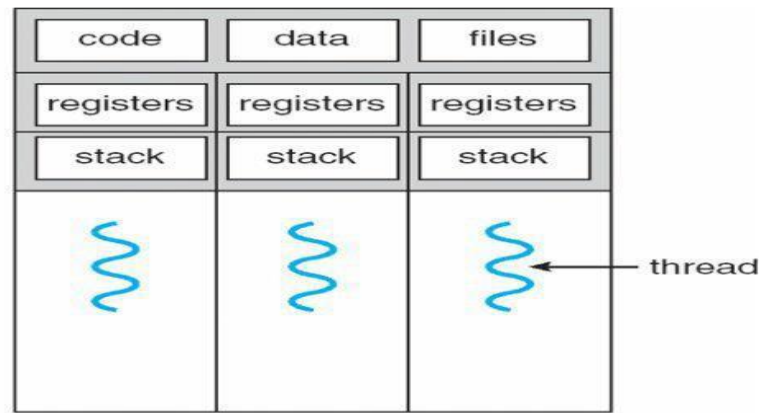
FILES- opening and closing files

REGISTER- contain information about CPU state

STACK- parameters, local variables, functions



single-threaded process



multithreaded process

Types Of Threads:

1) User Threads : Thread creation, scheduling, management happen in user space by Thread Library. user threads are faster to create and manage. If a user thread performs a system call, which blocks it, all the other threads in that process one also automatically blocked, whole process is blocked.

Advantages

- ☐ Thread switching does not require Kernel mode privileges.
- ☐ User level thread can run on any operating system.
- ☐ Scheduling can be application specific in the user level thread.
- ☐ User level threads are fast to create and manage.

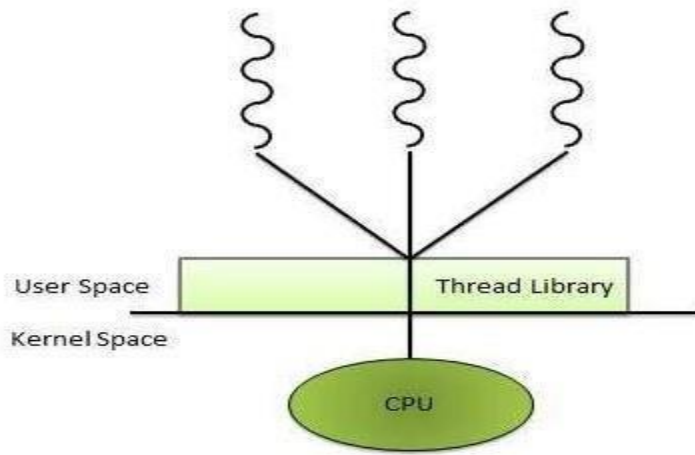
Disadvantages

- ☐ In a typical operating system, most system calls are blocking.
- ☐ Multithreaded application cannot take advantage of multiprocessing.

2) Kernel Threads: kernel creates, schedules, manages these threads .these threads are slower, manage. If one thread in a process blocked, over all process need not be blocked.

Advantages

- ☐ Kernel can simultaneously schedule multiple threads from the same process on multiple processes.
- ☐ If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- ☐ Kernel routines themselves can multithreaded.



Disadvantages

- ☐ Kernel threads are generally slower to create and manage than the userthreads.
- ☐ Transfer of control from one thread to another within same process requires a mode switch to the Kernel.

Multithreading Models

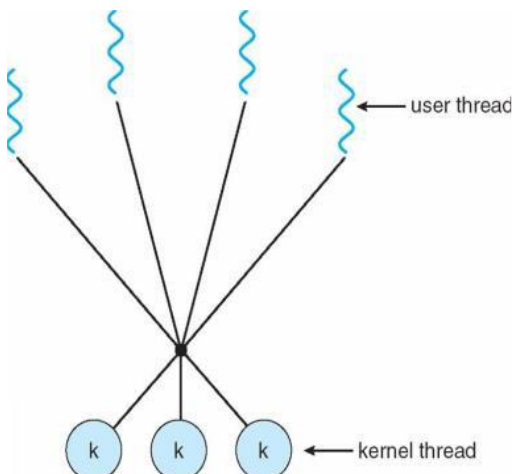
Some operating system provides a combined user level thread and Kernel level thread facility. Solaris is a good example of this combined approach. In a combined system, multiple threads within the same application can run in parallel on multiple processors and a blocking system call need not block the entire process. Multithreading models are three types

- ☐ Many to many relationship.
- ☐ Many to one relationship.
- ☐ One to one relationship.

Many to Many Model

In this model, many user level threads multiplexes to the Kernel thread of smaller or equal numbers. The number of Kernel threads may be specific to either a particular application or a particular machine.

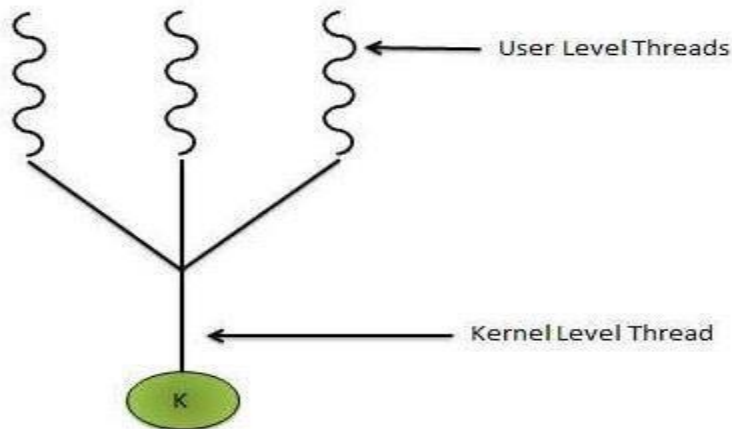
Following diagram shows the many to many model. In this model, developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallels on a multiprocessor.



Many to One Model

Many to one model maps many user level threads to one Kernel level thread. Thread management is done in user space. When thread makes a blocking system call, the entire process will be blocks. Only one thread can access the Kernel at a time,so multiple threads are unable to run in parallel on multiprocessors.

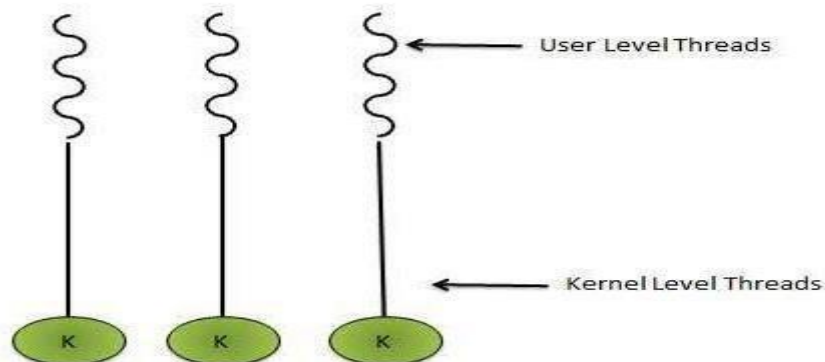
If the user level thread libraries are implemented in the operating system in such a way that system does not support them then Kernel threads use the many to one relationship modes.



One to One Model

There is one to one relationship of user level thread to the kernel level thread. This model provides more concurrency than the many to one model. It also another thread to run when a thread makes a blocking system call. It support multiple thread to execute in parallel on microprocessors.

Disadvantage of this model is that creating user thread requires the corresponding Kernel thread. OS/2, windows NT and windows 2000 use one to one relationship model.



Difference between User Level & Kernel Level Thread

S.N.	User Level Threads	Kernel Level Thread
1	User level threads are faster to create and manage.	Kernel level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User level thread is generic and can run on any operating system.	Kernel level thread is specific to the operating system.
4	Multi-threaded application cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

CPU Scheduling Algorithm:

CPU Scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated first to the CPU. There are four types of CPU scheduling that exist.

1. **First Come, First Served Scheduling (FCFS) Algorithm:** This is the simplest CPU scheduling algorithm. In this scheme, the process which requests the CPU first, that is allocated to the CPU first. The implementation of the FCFS algorithm is easily managed with a FIFO queue. When a process enters the ready queue its PCB is linked onto the rear of the queue. The average waiting time under FCFS policy is quite long. It is Non Primitive Scheduling Algorithm. Consider the following set of processes that arrive at time 0, the length of the cpu burst time given in milli seconds. burst time is the time, required the cpu to execute that job, it is in milli seconds.

Process	CPU time
P ₁	5
P ₂	24
P ₃	16
P ₄	10
P ₃	

Using FCFS algorithm find the average waiting time and average turnaround time if the order is P₁, P₂, P₃, P₄.

Solution: If the process arrived in the order P₁, P₂, P₃, P₄ then according to the FCFS the Gantt chart will be:

P1	P2	P3	P4	P5	
0	5	29	45	55	58

Average turn around time:

Turn around time= waiting time + burst time

Turn around time for p1= 0+5=5.

Turn around time for p2=5+24=29

Turn around time for p3=29+16=45

Turn around time for p4=45+10=55

Turn around time for p5= 55+3=58

Average turn around time= $(5+29+45+55+58)/5 = 187/5 = 37.5$ milliseconds

Average waiting time:

waiting time= starting time- arrival time

Waiting time for p1=0

Waiting time for p2=5-0=5

Waiting time for p3=29-0=29

Waiting time for p4=45-0=45

Waiting time for p5=55-0=55

Average waiting time= $0+5+29+45+55/5 = 125/5 = 25$ ms.

Average Response Time :

Formula : First Response - Arrival Time

Response Time for P1 =0

Response Time for P2 => 5-0 = 5

Response Time for P3 => 29-0 = 29

Response Time for P4 => 45-0 = 45

Response Time for P5 => 55-0 = 55

Average Response Time => $(0+5+29+45+55)/5 => 25$ ms

2.Shortest Job First Scheduling (SJF) Algorithm:

This algorithm associates with each process if the CPU is available. Which process having the smallest CPU burst time, CPU is assigned to that process . If two process having the same CPU burst time, FCFS is used. The SJF algorithm may be non preemptive algorithm. The preemptive SJF is also known as shortest remaining time first.

PROCESS	CPU BT
P1	5
P2	24
P3	16
P4	10
P5	3

P5	P1	P4	P3	P2	
0	3	8	18	34	58

P5 having the least CPU burst time (3ms). CPU assigned to that (P5). After completion of P5 short term scheduler search for next (P1).....

Average Waiting Time :

Formula = Starting Time - Arrival Time

waiting Time for P1 => $3-0 = 3$

waiting Time for P2 => $34-0 = 34$

waiting Time for P3 => $18-0 = 18$

waiting Time for P4 => $8-0=8$

waiting time for P5=0

Average waiting time => $(3+34+18+8+0)/5 => 63/5 =12.6 \text{ ms}$

Average Turn Around Time :

Formula = waiting Time + burst Time

Turn Around Time for P1 => $3+5 =8$

Turn Around for P2 => $34+24 =58$

Turn Around for P3 => $18+16 = 34$

Turn Around Time for P4 => $8+10 =18$

Turn Around Time for P5 => $0+3 = 3$

Average Turn around time => $(8+58+34+18+3)/5 => 121/5 = 24.2 \text{ ms}$

Average Response Time :

Formula : First Response - Arrival Time

First Response time for P1 => $3-0 = 3$

First Response time for P2 => $34-0 = 34$

First Response time for P3 => $18-0 = 18$

First Response time for P4 => $8-0 = 8$

First Response time for P5 = 0

Average Response Time => $(3+34+18+8+0)/5 => 63/5 = 12.6 \text{ ms}$ SJF is Non primitive scheduling algorithm

Advantages : Least average waiting time

Least average turn around time

Least average response time

Average waiting time (FCFS) = 25 ms

Average waiting time (SJF) = 12.6 ms 50% time saved in SJF.

Disadvantages:

- Knowing the length of the next CPU burst time is difficult.
- Aging (Big Jobs are waiting for long time for CPU)

3. **Shortest Remaining Time First (SRTF)**; This is primitive scheduling algorithm. Short term scheduler always chooses the process that has term shortest remaining time. When a new process joins the ready queue , short term scheduler compare the remaining time of executing process and new process. If the new process has the least CPU burst time, The scheduler selects that job and connect to CPU. Otherwise continue the old process.

process	BT	AT	CT	TAT=(CT-AT)	WT=(TAT-BT)	RESPONSE TIME=FR-AT
P1	3	0	3	3	0	0
P2	6	2	15	13	7	1
P3	4	4	8	4	0	0
P4	5	6	20	14	9	9
P5	2	8	10	2	0	0

P1	P2	P3	P5	P2	P4	
0	3	4	8	10	15	20

P1 arrives at time 0, P1 executing First ,

P2 arrives at time 2. Compare P1 remaining time and P2 ($3-2 = 1$) and 6. So, continue P1

after P1, executing P2, at time 4, P3 arrives, compare P2 remaining time ($6-1=5$) and 4 ($4<5$) .So, executing P3 at time 6, P4 arrives. Compare P3 remaining time and P4 ($4-2=2$) and 5 ($2<5$) . So, continue P3 , after P3, ready queue consisting P5 is the least out of three. So execute P5, next P2, P4.

3. **Priority Scheduling Algorithm**: In this scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority. Equal priority processes are scheduled in FCFS manner.

Consider the following example:

PROCESS	BURST TIME	PRIORITY	WT	TAT	
P1	6	2	$3-0=3$	$3+6=9$	
P2	12	4	$13-0=13$	$13+12=25$	
P3	1	5	$25-0=25$	$25+1=26$	
P4	3	1	0	$0+3=3$	
P5	4	3	$9-0=9$	$9+4=13$	

P4 has the highest priority. Allocate the CPU to process P4 first next P1, P5, P2, P3.

P4 has the highest priority, so it is scheduled first, then P1, P2, P3, P5.					
P4	P1	P5	P2	P3	
0	3	9	13	25	26

4. **Round Robin Scheduling Algorithm:** It is designed especially for time sharing systems. Here CPU switches between the processes. When the time quantum expired, the CPU switched to another job. A small unit of time, called a time quantum or time slice. A time quantum is generally from 10 to 100 ms. The time quantum is generally depending on OS. Here ready queue is a circular queue. CPU scheduler picks the first process from ready queue, sets timer to interrupt after one time quantum and dispatches the process.

PROCESS	BURST TIME	WT	TAT	
P1	30	$0 + (15 - 5) + (24 - 20) = 14$	$14 + 30 = 44$	
P2	6	$5 + (20 - 10) = 15$	$15 + 6 = 21$	
P3	8	$10 + (21 - 15) - 0 = 16$	$16 + 8 = 24$	

Time Slice = 5 millisecond.

P1	P2	P3	P1	P2	P3	P1	P1	P1	P1	
0	5	10	15	20	21	24	29	34	39	44

Multilevel Queue Scheduling Algorithm

Another class of scheduling algorithms has been created for situations in which processes are easily classified into different groups.

For example, A common division is made between foreground(or interactive) processes and background (or batch) processes. These two types of processes have different response-time requirements, and so might have different scheduling needs. In addition, foreground processes may have priority over background processes.

A multi-level queue scheduling algorithm partitions the ready queue into several separate queues. The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority, or process type. Each queue has its own scheduling algorithm.

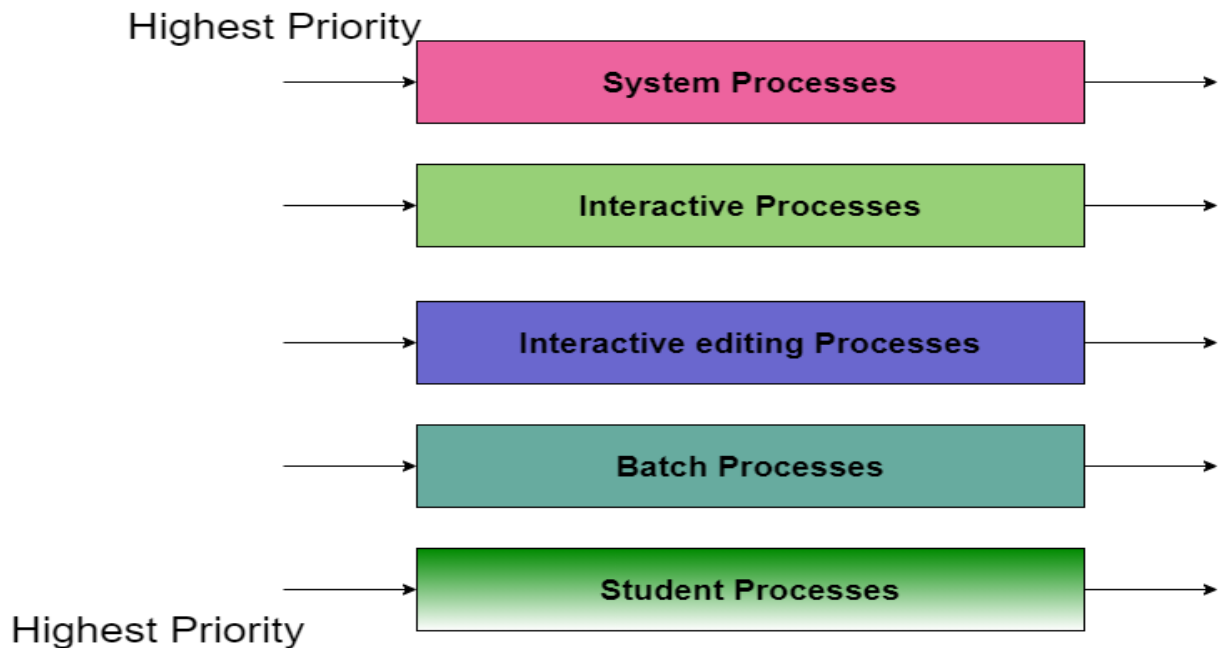
For example, separate queues might be used for foreground and background processes. The foreground queue might be scheduled by the Round Robin algorithm, while the background queue is scheduled by an FCFS algorithm.

In addition, there must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling. **For example,** The foreground queue may have absolute priority over the background queue.

Let us consider an example of a multilevel queue-scheduling algorithm with five queues:

1. System Processes
2. Interactive Processes
3. Interactive Editing Processes
4. Batch Processes
5. Student Processes

Each queue has absolute priority over lower-priority queues. No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty. If an interactive editing process entered the ready queue while a batch process was running, the batch process will be preempted.



In this case, if there are no processes on the higher priority queue only then the processes on the low priority queues will run. For **Example:** Once processes on the system queue, the Interactive queue, and Interactive editing queue become empty, only then the processes on the batch queue will run.

The Description of the processes in the above diagram is as follows:

- **System Process** The Operating system itself has its own process to run and is termed as System Process.
- **Interactive Process** The Interactive Process is a process in which there should be the same kind of interaction (basically an online game).
- **Batch Processes** Batch processing is basically a technique in the Operating system that collects the programs and data together in the form of the **batch** before the **processing** starts.
- **Student Process** The system process always gets the highest priority while the student processes always get the lowest priority.

In an operating system, there are many processes, in order to obtain the result we cannot put all processes in a queue; thus this process is solved by Multilevel queue scheduling.

With the help of this scheduling we can apply various kind of scheduling for different kind of processes:

For System Processes: First Come First Serve(FCFS) Scheduling.

For Interactive Processes: Shortest Job First (SJF) Scheduling.

For Batch Processes: Round Robin(RR) Scheduling

For Student Processes: Priority Scheduling

Multilevel Feedback Queue Scheduling

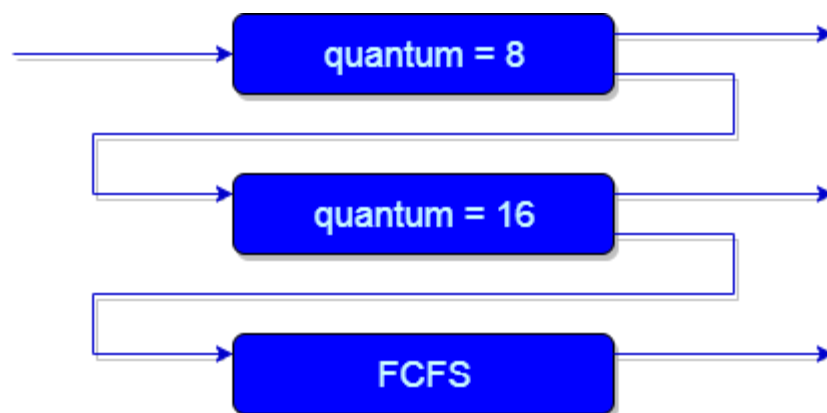
In a multilevel queue-scheduling algorithm, processes are permanently assigned to a queue on entry to the system. Processes do not move between queues. This setup has the advantage of low scheduling overhead, but the disadvantage of being inflexible.

Multilevel feedback queue scheduling, however, allows a process to move between queues. The idea is to separate processes with different CPU-burst characteristics. If a process uses too much CPU time, it will be moved to a lower-priority queue. Similarly, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.

In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues.
- The scheduling algorithm for each queue.
- The method used to determine when to upgrade a process to a higher-priority queue.
- The method used to determine when to demote a process to a lower-priority queue.
- The method used to determine which queue a process will enter when that process needs service.

The definition of a multilevel feedback queue scheduler makes it the most general CPU-scheduling algorithm. It can be configured to match a specific system under design. Unfortunately, it also requires some means of selecting values for all the parameters to define the best scheduler. Although a multilevel feedback queue is the **most general scheme**, it is also the **most complex**.



An example of a multilevel feedback queue can be seen in the above figure.

Explanation:

First of all, Suppose that queues 1 and 2 follow round robin with time quantum 8 and 16 respectively and queue 3 follows FCFS. One of the implementations of Multilevel Feedback Queue Scheduling is as follows:

1. If any process starts executing then firstly it enters queue 1.
2. In queue 1, the process executes for 8 unit and if it completes in these 8 units or it gives CPU for I/O operation in these 8 units unit than the priority of this process does not change, and if for some reasons it again comes in the ready queue than it again starts its execution in the Queue 1.
3. If a process that is in queue 1 does not complete in 8 units then its priority gets reduced and it gets shifted to queue 2.
4. Above points 2 and 3 are also true for processes in queue 2 but the time quantum is 16 units. Generally, if any process does not complete in a given time quantum then it gets shifted to the lower priority queue.
5. After that in the last queue, all processes are scheduled in an FCFS manner.
6. It is important to note that a process that is in a lower priority queue can only execute only when the higher priority queues are empty.
7. Any running process in the lower priority queue can be interrupted by a process arriving in the higher priority queue.

Also, the above implementation may differ for the example in which the last queue will follow **Round-robin Scheduling**.

In the above Implementation, there is a problem and that is; Any process that is in the lower priority queue has to suffer starvation due to some short processes that are taking all the CPU time.

And the solution to this problem is : There is a solution that is to boost the priority of all the process after regular intervals then place all the processes in the highest priority queue.

The need for Multilevel Feedback Queue Scheduling(MFQS)

Following are some points to understand the need for such complex scheduling:

- This scheduling is more flexible than Multilevel queue scheduling.
- This algorithm helps in reducing the response time.
- In order to optimize the turnaround time, the SJF algorithm is needed which basically requires the running time of processes in order to schedule them. As we know that the running time of processes is not known in advance. Also, this scheduling mainly runs a process for a time quantum and after that, it can change the priority of the process if the process is long. Thus this scheduling algorithm mainly learns from the past behavior of the processes and then it can predict the future behavior of the processes. In this way, MFQS tries to run a shorter process first which in return leads to optimize the turnaround time.

Advantages of MFQS

- This is a flexible Scheduling Algorithm

- This scheduling algorithm allows different processes to move between different queues.
- In this algorithm, A process that waits too long in a lower priority queue may be moved to a higher priority queue which helps in preventing starvation.

Disadvantages of MFQS

- This algorithm is too complex.
- As processes are moving around different queues which leads to the production of more CPU overheads.
- In order to select the best scheduler this algorithm requires some other means to select the values

Multiple-Processor Scheduling:

In multiple-processor scheduling **multiple CPU's** are available and hence **Load Sharing** becomes possible.

Load sharing resolves around balancing the load between multiple processors. However multiple processor scheduling is more **complex** as compared to single processor scheduling. Multi processor systems may be heterogeneous (It contains different kinds of CPU's) or Homogeneous(all the same kind of CPU). we can use any processor available to run any process in the queue.

Approaches to Multiple-Processor Scheduling –

One approach is when all the scheduling decisions and I/O processing are handled by a single processor which is called the **Master Server** and the other processors executes only the **user code**. This is simple and reduces the need of data sharing. This entire scenario is called **Asymmetric Multiprocessing**.

A second approach uses **Symmetric Multiprocessing** where each processor is **self scheduling**. All processes may be in a common ready queue or each processor may have its own private queue for ready processes. The scheduling proceeds further by having the scheduler for each processor examine the ready queue and select a process to execute.

Processor Affinity –

Processor Affinity means a processes has an **affinity** for the processor on which it is currently running.

When a process runs on a specific processor there are certain effects on the cache memory. The data most recently accessed by the process populate the cache for the processor and as a result successive memory access by the process are often satisfied in the cache memory. Now if the process migrates to another processor, the contents of the cache memory must be invalidated for the first processor and the cache for the second processor must be repopulated. Because of the high cost of invalidating and repopulating caches, most of the

SMP(symmetric multiprocessing) systems try to avoid migration of processes from one processor to another and try to keep a process running on the same processor. This is known as **PROCESSOR AFFINITY**.

There are two types of processor affinity:

1. **Soft Affinity** – When an operating system has a policy of attempting to keep a process running on the same processor but not guaranteeing it will do so, this situation is called soft affinity.
2. **Hard Affinity** – Hard Affinity allows a process to specify a subset of processors on which it may run. Process specifies that it is not to be moved between processors.

Load Balancing –

Load Balancing is the **phenomena** which keeps the **workload** evenly **distributed** across all processors in an SMP system. Load balancing is necessary only on systems where each processor has its own private queue of process which are eligible to execute. Load balancing is unnecessary because once a processor becomes idle it immediately extracts a runnable process from the common run queue. On SMP(symmetric multiprocessing), it is important to keep the workload balanced among all processors to fully utilize the benefits of having more than one processor else one or more processor will sit idle while other processors have high workloads along with lists of processors awaiting the CPU.

There are two general approaches to load balancing :

1. **Push Migration** – In push migration a task routinely checks the load on each processor and if it finds an imbalance then it evenly distributes load on each processors by moving the processes from overloaded to idle or less busy processors.
2. **Pull Migration** – Pull Migration occurs when an idle processor pulls a waiting task from a busy processor for its execution.

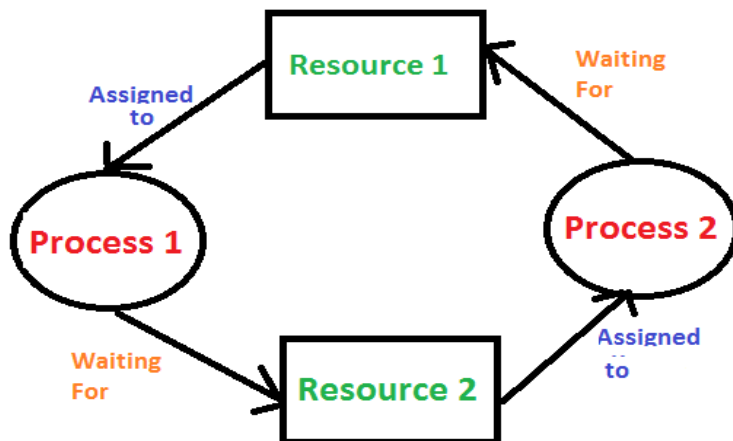
Deadlock:

Definition: A set of processes is in a deadlock state if every process in the set is waiting for an event (release) that can only be caused by some other process in the same set.

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Consider an example when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other. A similar situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by

other(s). For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



System model:

A system consists of a finite number of resources to be distributed among a number of competing processes. The resources are partitioned into several types, each consisting of some number of identical instances. Memory space, CPU cycles, files, I/O devices are examples of resource types. If a system has 2 CPUs, then the resource type CPU has 2 instances.

A process must request a resource before using it and must release the resource after using it. A process may request as many resources as it requires to carry out its task. The number of resources as it requires to carry out its task. The number of resources requested may not exceed the total number of resources available in the system. A process cannot request 3 printers if the system has only two.

A process may utilize a resource in the following sequence:

- (I) REQUEST: The process requests the resource. If the request cannot be granted immediately (if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
- (II) USE: The process can operate on the resource .if the resource is a printer, the process can print on the printer.
- (III) RELEASE: The process release the resource.

For each use of a kernel managed by a process the operating system checks that the process has requested and has been allocated the resource. A system table records whether each resource is free (or) allocated. For each resource that is allocated, the table also records the process to which it is allocated. If a process requests a resource that is currently allocated to another process, it can be added to a queue of processes waiting for this resource.

Starvation vs Deadlock

Starvation	Deadlock
When all the low priority processes got blocked, while the high priority processes execute then this situation is termed as Starvation.	Deadlock is a situation that occurs when one of the processes got blocked.
Starvation is a long waiting but it is not an infinite process.	Deadlock is an infinite process.
It is not necessary that every starvation is a deadlock.	There is starvation in every deadlock.
Starvation is due to uncontrolled priority and resource management.	During deadlock, preemption and circular wait does not occur simultaneously.

DEADLOCK CHARACTERIZATION:

In a deadlock, processes never finish executing, and system resources are tied up, preventing other jobs from starting.

Necessary Conditions

The deadlock situation can only arise if all the following four conditions hold simultaneously

- 1. MUTUAL EXCLUSION:** Only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
- 2. HOLD AND WAIT:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
- 3. NO PREEMPTION:** Resources cannot be preempted. A resource can be released only voluntarily by the process holding it, after that process has completed its task.
- 4. CIRCULAR WAIT:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for resource held by P_1 , P_1 is waiting for a resource held by $P_2, \dots, P_{n-1}$ is waiting for a resource held by P_n and P_n is waiting for a resource held by P_0 .

The above four conditions are not completely independent as the circular wait condition implies the hold and wait condition. We emphasize on the fact that all four conditions must hold for a deadlock.

RESOURCE ALLOCATION GRAPH

Deadlocks can be described more precisely in terms of a directed graph called a system resource allocation graph. This graph consists of a set of vertices V and a set of edges E . the set of vertices V is partitioned into 2 different types of nodes:

$P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system. $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.

A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$. It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$, it signifies that an instance of resource type R_j has been allocated to process P_i .

A directed edge $P_i \rightarrow R_j$ is called a requested edge. A directed edge $R_j \rightarrow P_i$ is called an assignment edge.

We represent each process P_i as a circle, each resource type R_j as a rectangle. Since resource type R_j may have more than one instance. We represent each such instance as a dot within the rectangle. A request edge points to only the rectangle R_j . An assignment edge must also designate one of the dots in the rectangle.

When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource allocation graph. When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge. When the process no longer needs access to the resource, it releases the resource, as a result, the assignment edge is deleted.

The sets P , R , E :

$P = \{P_1, P_2, P_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

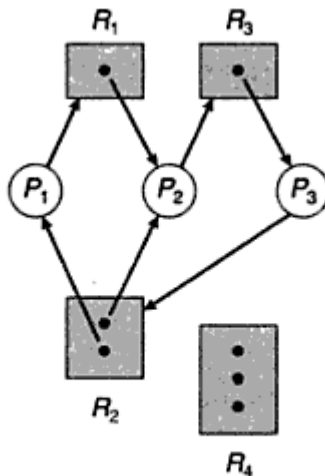


Figure Resource-allocation graph with a deadlock.

One instance of resource type R_1

Two instances of resource type R_2

One instance of resource type R_3

Three instances of resource type R_4

PROCESS STATES:

Process P_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1 .

Process P2 is holding an instance of R1 and an instance of R2 and is waiting for instance of R3. Process P3 is holding an instance of R3.

If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.

Suppose that process P3 requests an instance of resource type R2. Since no resource instance is currently available, a request edge P3 → R2 is added to the graph.

2 cycles:

P1 → R1 → P2 → R3 → P3 → R2 → P1

P2 → R3 → P3 → R2 → P2

Processes P1, P2, P3 are deadlocked. Process P2 is waiting for the resource R3, which is held by process P3. Process P3 is waiting for either process P1 (or) P2 to release resource R2. In addition, process P1 is waiting for process P2 to release resource R1.

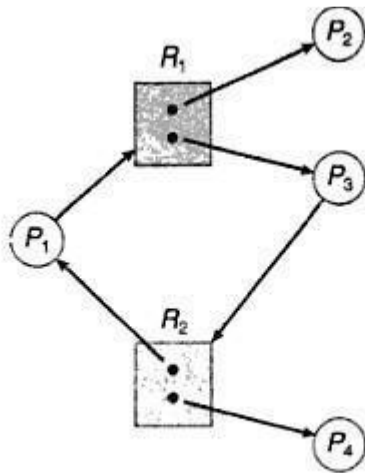


Figure Resource-allocation graph with a cycle but no deadlock.

We also have a cycle: P1 → R1 → P3 → R2 → P1

However there is no deadlock. Process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.

Methods For Handling Deadlocks

Methods that are used in order to handle the problem of deadlocks are as follows:

1. Ignoring the Deadlock

According to this method, it is assumed that deadlock would never occur. This approach is used by many operating systems where they assume that deadlock will never occur which means operating systems simply ignore the deadlock. This approach can be beneficial for those systems that are only used for browsing and for

normal tasks. Thus ignoring the deadlock method can be useful in many cases but it is not perfect in order to remove the deadlock from the operating system.

2. Deadlock Prevention

For a deadlock to occur, each of the 4 necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

Mutual Exclusion:

not required for sharable resources; must hold for non sharable resources. Shared resources such as read-only files do not lead to deadlocks. Unfortunately some resources, such as printers and tape drives, require exclusive access by a single process.

Hold and Wait

To prevent this condition processes must be prevented from holding one or more resources while simultaneously waiting for one or more others. There are several possibilities for this:

- i. Require that all processes request all resources at one time. This can be wasteful of system resources if a process needs one resource early in its execution and doesn't need some other resource until much later.
- ii. Require that processes holding resources must release them before requesting new resources, and then reacquire the released resources along with the new ones in a single new request. This can be a problem if a process has partially completed an operation using a resource and then fails to get it re-allocated after releasing it.
- iii. Either of the methods described above can lead to starvation if a process requires one or more popular resources.

No Preemption

Preemption of process resource allocations can prevent this condition of deadlocks, when it is possible.

- i. One approach is that if a process is forced to wait when requesting a new resource, then all other resources previously held by this process are implicitly released, (preempted), forcing this process to reacquire the old resources along with the new resources in a single request.
- ii. Another approach is that when a resource is requested and not available, then the system looks to see what other processes currently have those resources and are themselves blocked waiting for some other resource. If such a process is found, then some of their resources may get preempted and added to the list of resources for which the process is waiting.
- iii. Either of these approaches may be applicable for resources whose states are easily saved and restored, such as registers and memory, but are generally not applicable to other devices such as printers and tape drives.

Circular Wait

- i. One way to avoid circular wait is to number all resources, and to require that processes request resources only in strictly increasing (or decreasing) order.
- ii. In other words, in order to request resource R_j , a process must first release all R_i such that $i \geq j$.

3. Deadlock Avoidance

The general idea behind deadlock avoidance is to prevent deadlocks from ever happening, by preventing at least one of the aforementioned conditions. This requires more information about each process, AND tends to lead to low device utilization. (I.e. it is a conservative approach.)

In some algorithms the scheduler only needs to know the maximum number of each resource that a process might potentially use. In more complex algorithms the scheduler can also take advantage of the schedule of exactly what resources may be needed in what order.

When a scheduler sees that starting a process or granting resource requests may lead to future deadlocks, then that process is just not started or the request is not granted.

A resource allocation state is defined by the number of available and allocated resources, and the maximum requirements of all processes in the system.

Safe & Unsafe State

- i. A state is safe if the system can allocate all resources requested by all processes (up to their stated maximums) without entering a deadlock state.
- ii. More formally, a state is safe if there exists a safe sequence of processes $\{ P_0, P_1, P_2, \dots, P_N \}$ such that all of the resource requests for P_i can be granted using the resources currently allocated to P_i and all processes P_j where $j < i$. (I.e. if all the processes prior to P_i finish and free up their resources, then P_i will be able to finish also, using the resources that they have freed up.)
- iii. If a safe sequence does not exist, then the system is in an unsafe state, which MAY lead to deadlock. (All safe states are deadlock free, but not all unsafe states lead to deadlocks.)

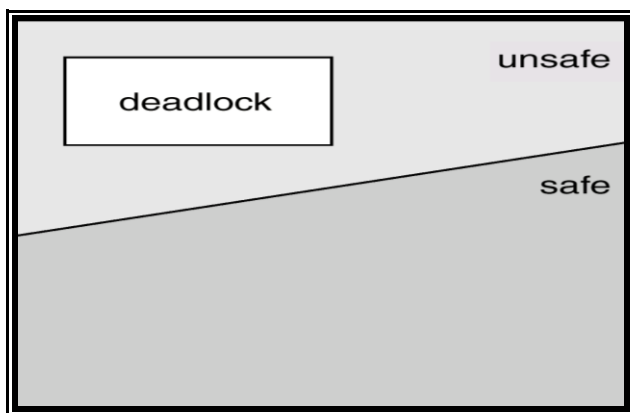


Fig: Safe, unsafe, and deadlocked state spaces.

Avoidance algorithms

Single instance of a resource type

- o Use a resource-allocation graph
- Multiple instances of a resource type
- o Use the banker's algorithm

Resource-Allocation Graph Scheme

Example 1 Consider a system with 12 tape drives. Assume there are three processes : p1, p2, p3. Assume we know the maximum number of tape drives that each process may request:

p1 : 10, p2 : 4, p3 : 9

Suppose at time t_{now} , 9 tape drives are allocated as follows : p1 : 5, p2 : 2, p3 : 2 So, we have three more tape drives which are free. This system is in a safe state because if we sequence processes as: <p2,p1,p3> then p2 can get two more tape drives and it finishes its job, and returns four tape drives to the system. Then the system will have 5 free tape drives. Allocate all of them to p1, it gets 10 tape drives and finishes its job. p1 then returns all 10 drives to the system. Then p3 can get 7 more tape drives and it does its job.

It is possible to go from a safe state to an unsafe state:

Example 2

Consider the above example. At time $t_{\text{now}+1}$, p3 requests one more tape drive and gets it. Now, the system is in an unsafe state. There are two free tape drives, so only p2 can be allocated all its tape drives. When it finishes and returns all 4 tape drives, the system will have four free tape drives. p1 is allocated 5, may request 5 more → has to wait p3 is allocated 3, may request 6 more → has to wait We allocated p3 one more tape drive and this caused a deadlock.

Banker's Algorithm

- For resource categories that contain more than one instance the resource-allocation graph method does not work, and more complex (and less efficient) methods must be chosen.
- The Banker's Algorithm gets its name because it is a method that bankers could use to assure that when they lend out resources they will still be able to satisfy all their clients. (A banker won't loan out a little money to start building a house unless they are assured that they will later be able to loan out the rest of the money to finish the house.)
- When a process starts up, it must state in advance the maximum allocation of resources it may request, up to the amount available on the system.
- When a request is made, the scheduler determines whether granting the request would leave the system in a safe state. If not, then the process must wait until the request can be granted safely.

Banker's Algorithm

It is a deadlock avoidance algorithm.

The following data structures are used in the algorithm:

m = number of resources

n = number of processes

Available [m] : One dimensional array of size m . It indicates the number of available resources of each type. For example, if Available [i] is k , there are k instances of resource r_i .

Max [n, m] : Two dimensional array of size $n \times m$. It defines the maximum demand of each process from each resource type. For example, if Max [i, j] is k , process p_i may request at most k instances of resource type r_j .

Allocation [n, m] : Two dimensional array of size $n \times m$. It defines the number of resources of each type currently allocated to each process.

Need [n, m] : Two dimensional array of size $n \times m$. It indicates the remaining need of each process, of each resource type. If Need [i, j] is k , process p_i may need k more instances of resource type r_j . Note that Need [i, j] = Max [i, j] - Allocation [i, j]. Request [n, m] : Two dimensional array of size $n \times m$. It indicates the pending requests of each process, of each resource type.

Now, take each row vector in Allocation and Need as Allocation(i) and Need(i). (Allocation(i) specifies the resources currently allocated to process p_i .)

Define the $\leq$ relation between two vectors X and Y , of equal size = n as :

$$X \leq Y \Leftrightarrow X[i] \leq Y[i], i = 1, 2, \dots, n$$

$$X \not\leq Y \Leftrightarrow X[i] > Y[i] \text{ for some } i$$

The algorithm is as follows:

1. Process p_i makes requests for resources. Let Request(i) be the corresponding request vector. So, if p_i wants k instances of resource type r_j , then Request(i)[j] = k .
2. If Request(i) $\not\leq$ Need(i), there is an error.
3. Otherwise, if Request(i) $\not\leq$ Available, then p_i must wait.
4. Otherwise, Modify the data structures as follows :

$$\text{Available} = \text{Available} - \text{Request}(i)$$

$$\text{Allocation}(i) = \text{Allocation}(i) + \text{Request}(i)$$

$$\text{Need}(i) = \text{Need}(i) - \text{Request}(i)$$

5. Check whether the resulting state is safe. (Use the safety algorithm presented below.)

6. If the state is safe, do the allocation. Otherwise, p_i must wait for Request(i).

Safety Algorithm to perform Step 5:

Let Work and Finish be vectors of length m and n , respectively.

1. Initialize Work = Available, Finish [j] = false, for all j .

2. Find an i such that Finish [i] = false and Need(i) $\leq$ Work If no such i is found, go to step 4.

3. If an i is found, then for that i , do :

Work = Work + Allocation(i)

Finish [i] = true

Go to step 2.

4. If Finish [j] = true for all j , then the system is in a safe state.

Banker's algorithm time complexity is $O(m \times (n^2))$.

Example: Consider the following snapshot at time T_0 of the system and answer the following questions using Banker's Algorithm [2015-16]

I. Compute the total number of resources of each type.

II. Compute the need matrix

III. Is the system in a Safe state?

IV. If a request from P_1 arrives for (0, 4, 2, 0), can the request granted immediately?

PROCESS	ALLOCATION(A B C D)	MAX (A B C D)	AVAILABLE (A B C D)
P0	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 6 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

Solution:

I) Total no of resources of each type can be obtained by adding Allocation(j)+Available

$$A = 0+1+1+0+0+1 = 3$$

$$B = 0+0+3+6+0+5 = 14$$

$$C = 1+0+5+3+1+2 = 12$$

$$D = 2+0+4+2+4+0 = 12$$

Hence total no of resources = A B C D

3 14 12 12

II) Need matrix is calculated by subtracting Allocation Matrix from the Max matrix

	Need(Max-Allocation)			
	A	B	C	D
P ₀	0	0	0	0
P ₁	0	7	5	0
P ₂	1	0	0	2
P ₃	0	0	2	0
P ₄	0	6	4	2

III) To check if system is in a safe state

- The Available matrix is [1520].
- A process after it has finished execution is supposed to free up all the resources it hold.
- We need to find a safety sequence such that it satisfies the criteria $\text{Need} \leq \text{Available}$.
- Since $\text{Need}(P_0) \leq \text{Available}$, we select P₀. $[\text{Available}] = [\text{Available}] + [\text{Allocation}(P_0)]$

$\text{Available} = [1520] + [0012] = [1532]$

- $\text{Need}(P_2) \leq \text{Available} \rightarrow \text{Available} = [1\ 5\ 3\ 2] + [1\ 3\ 5\ 4] = [2\ 8\ 8\ 6]$
- $\text{Need}(P_3) \leq \text{Available} \rightarrow \text{Available} = [2\ 8\ 8\ 6] + [0\ 6\ 3\ 2] = [2\ 14\ 11\ 8]$
- $\text{Need}(P_4) \leq \text{Available} \rightarrow \text{Available} = [2\ 14\ 11\ 8] + [0\ 0\ 1\ 4] = [2\ 14\ 12\ 12]$
- $\text{Need}(P_1) \leq \text{Available} \rightarrow \text{Available} = [2\ 14\ 12\ 12] + [1\ 0\ 0\ 0] = [3\ 14\ 12\ 12]$

Safe Sequence is <p₀,p₂,p₃,p₄,p₁>

IV) A request from process P₁ arrives for (0,4,2,0)

- System receives a request for P₁ for Req(P₁)[0420]
- First we check if Req(P₁) is less than Need(P₁) $\rightarrow [0420] < [0750]$ is true
- Now we check if Req(P₁) is less than Available $\rightarrow [0420] < [1520]$ is true.
- So we update the values as:

○ $\text{Available} = \text{Available} - \text{Request} = [1520] - [0420] = [1100]$

$\text{Allocation} = \text{allocation}(P_1) + \text{Request} = [1000] + [0420] = [1420]$

$\text{Need} = \text{Need}(P_1) - \text{Request} = [0750] - [0420] = [0330]$

	Allocation				Max				Need				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P ₀	0	0	1	2	0	0	1	2	0	0	0	0	1	1	0	0
P ₁	1	4	2	0	1	7	5	0	0	3	3	0				
P ₂	1	3	5	4	2	3	5	6	1	0	0	2				
P ₃	0	6	3	2	0	6	5	2	0	0	2	0				
P ₄	0	0	1	4	0	6	5	6	0	6	4	2				

- This is the modified table
- On verifying, we see that the safe sequence still remains the same .The system continues to remain in a safe state.

Example:

Let us consider the following snapshot for understanding the banker's algorithm:

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

1. Calculate the content of the need matrix?
2. Check if the system is in a safe state?
3. Determine the total sum of each type of resource?

Solution:

1. The Content of the need matrix can be calculated by using the formula given below:

Need = Max – Allocation

2. Let us now check for the safe state.

Safe sequence:

1. For process P0, Need = (3, 2, 1) and

Available = (2, 1, 0)

Need <= Available = False

So, the system will move to the next process.

2. For Process P1, Need = (1, 1, 0)

Available = (2, 1, 0)

Need <= Available = True

Request of P1 is granted.

Available = Available + Allocation

= (2, 1, 0) + (2, 1, 2)

= (4, 2, 2) (New Available)

3. For Process P2, Need = (5, 0, 1)

Available = (4, 2, 2)

Need <= Available = False

So, the system will move to the next process.

4. For Process P3, Need = (7, 3, 3)

Available = (4, 2, 2)

Need <= Available = False

So, the system will move to the next process.

5. For Process P4, Need = (0, 0, 0)

Available = (4, 2, 2)

Need <= Available = True

Request of P4 is granted.

Available = Available + Allocation

= (4, 2, 2) + (1, 1, 2)

= (5, 3, 4) now, (New Available)

6. Now again check for Process P2, Need = (5, 0, 1)

Available = (5, 3, 4)

Need $\leq$ Available = True

Request of P2 is granted.

Available = Available + Allocation

= (5, 3, 4) + (4, 0, 1)

= (9, 3, 5) now, (New Available)

7. Now again check for Process P3, Need = (7, 3, 3)

Available = (9, 3, 5)

Need $\leq$ Available = True

The request for P3 is granted.

Available = Available + Allocation

= (9, 3, 5) + (0, 2, 0) = (9, 5, 5)

8. Now again check for Process P0, = Need (3, 2, 1)

= Available (9, 5, 5)

Need $\leq$ Available = True

So, the request will be granted to P0.

Safe sequence: < P1, P4, P2, P3, P0 >

The system allocates all the needed resources to each process. So, we can say that the system is in a safe state.

3. The total amount of resources will be calculated by the following formula:

The total amount of resources = sum of columns of allocation + Available

= [8 5 7] + [2 1 0] = [10 6 7]

Deadlock Detection

i. If deadlocks are not avoided, then another approach is to detect when they have occurred and recover somehow.

ii. In addition to the performance hit of constantly checking for deadlocks, an algorithm must be in place for recovering from deadlocks, and there is potential for lost work when processes must be aborted or have their resources preempted.

There are two types of Detection algorithm.

1. Single Instance of Each Resource Type

- If each resource category has a single instance, then we can use a variation of the resource-allocation graph known as a wait-for graph.
- A wait-for graph can be constructed from a resource-allocation graph by eliminating the resources and collapsing the associated edges.
- An arc from P_i to P_j in a wait-for graph indicates that process P_i is waiting for a resource that process P_j is currently holding.

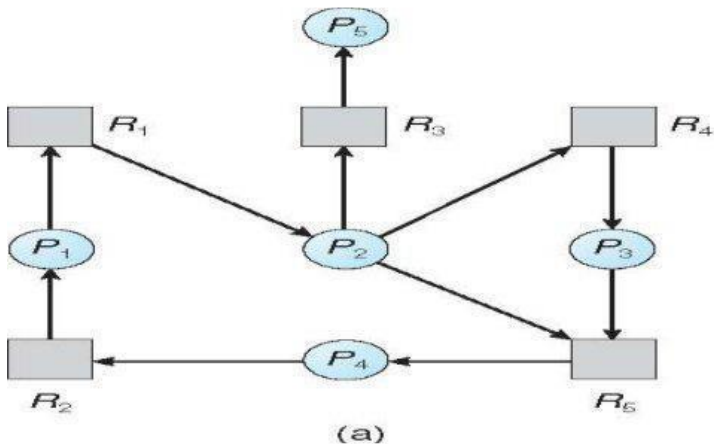


Fig: Resource Allocation Graph
Wait for Graph

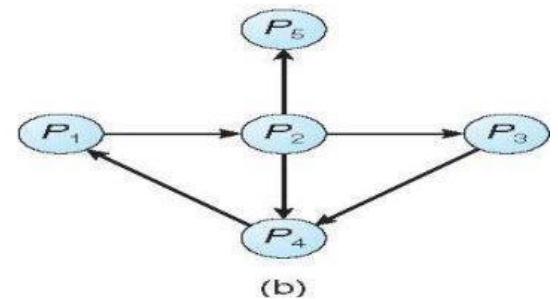


Fig: Corresponding

As before, cycles in the wait-for graph indicate deadlocks. This algorithm must maintain the wait-for graph, and periodically search it for cycles.

2. Several Instances of a Resource Type Available:

Available: A vector of length m indicates the number of available resources of each type.

Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.

Request: An $n \times m$ matrix indicates the current request of each process. If $\text{Request}[ij] = k$, then process P_i is requesting k more instances of resource type. R_j .

Detection Algorithm

1. Let Work and Finish be vectors of length m and n , respectively Initialize:

(a) $\text{Work} = \text{Available}$ (b) For $i = 1, 2, \dots, n$,

if $\text{Allocation}_i \neq 0$, then

$\text{Finish}[i] = \text{false}$;

otherwise,

$\text{Finish}[i] = \text{true}$

2. Find an index i such that both:

(a) $Finish[i] == false$

(b) $Request_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$

$Finish[i] = true$ go to step 2

4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state.

Moreover, if $Finish[i] == false$, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Example of Detection Algorithm

Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)

Snapshot at time T_0 :

	<i>Allocation</i>	<i>Need</i>	<i>Available</i>
	<i>ABC</i>	<i>ABC</i>	<i>ABC</i>
<i>P0</i>	010	000	000
<i>P1</i>	200	202	
<i>P2</i>	303	000	
<i>P3</i>	211	100	
<i>P4</i>	002	002	

Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = true$ for all i

P_2 requests an additional instance of type C

<u>Request</u>	<i>A</i>	<i>B</i>	<i>C</i>
<i>P0</i>	0	0	0
<i>P1</i>	2	0	1
<i>P2</i>	0	0	1
<i>P3</i>	1	0	0
<i>P4</i>	0	0	2

State of system?

Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests Deadlock exists, consisting of processes P_1, P_2, P_3 , and P_4

Deadlock recovery: There are two approaches for deadlock recovery.

1) Optimistic Approach (Preemption of Resources and processes)

- Preempt some resources from process & give these resources to other processes until the deadlock cycle is broken

2) Pessimistic Approach (Process termination)

- Abort all deadlock processes -----but it is very costly

- Abort one process at a time and decide next to abort after deadlock detection—but overhead of calling detection again and again and again

Process Termination

To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.

- Abort all deadlocked processes. This method clearly will break the deadlock cycle, but at great expense; the deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.
- Abort one process at a time until the deadlock cycle is eliminated. This method incurs considerable overhead, since, after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked. Aborting a process may not be easy. If the process was in the midst of updating a file, terminating it will leave that file in an incorrect state. Similarly, if the process was in the midst of printing data on a printer, the system must reset the printer to a correct state before printing the next job. If the partial termination method is used, then we must determine which deadlocked process (or processes) should be terminated. This determination is a policy decision, similar to CPU-scheduling decisions. The question is basically an economic one; we should abort those processes whose termination will incur the minimum cost. Unfortunately, the term minimum cost is not a precise one. Many factors may affect which process is chosen, including:

1. What the priority of the process is.
2. How long the process has computed and how much longer the process will compute before completing its designated task.
3. How many and what type of resources the process has used (for example, whether the resources are simple to preempt).
4. How many more resources the process needs in order to complete.
5. How many processes will need to be terminated.
6. Whether the process is interactive or batch.

Resource Preemption

To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken. If preemption is required to deal with deadlocks, then three issues need to be addressed:

- 1. Selecting a victim.** Which resources and which processes are to be preempted? As in process termination, we must determine the order of preemption to minimize cost. Cost factors may include such parameters as the number of resources a deadlocked process is holding and the amount of time the process has thus far consumed during its execution.

2. Rollback. If we preempt a resource from a process, what should be done with that process? Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state and restart it from that state. Since, in general, it is difficult to determine what a safe state is, the simplest solution is a total rollback: Abort the process and then restart it. Although it is more effective to roll back the process only as far as necessary to break the deadlock, this method requires the system to keep more information about the state of all running processes.

3. Starvation. How do we ensure that starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process? In a system where victim selection is based primarily on cost factors, it may happen that the same process is always picked as a victim. As a result, this process never completes its designated task, a starvation situation that must be dealt with in any practical system. Clearly, we must ensure that a process can be picked as a victim only a (small) finite number of times. The most common solution is to include the number of rollbacks in the cost factor.

Virtual machine

A **Virtual Machine** (VM) is a compute resource that uses software instead of a physical computer to run programs and deploy apps. Each virtual machine runs its own operating system and functions separately from the other VMs, even when they are all running on the same host. This means that, for example, a virtual MacOS virtual machine can run on a physical PC.

Virtual machine technology is used for many use cases across on-premises and cloud environments. More recently, public cloud services are using virtual machines to provide virtual application resources to multiple users at once, for even more cost efficient and flexible compute.

Use of virtual machine

Virtual machines (VMs) allow a business to run an operating system that behaves like a completely separate computer in an app window on a desktop. VMs may be deployed to accommodate different levels of processing power needs, to run software that requires a different operating system, or to test applications in a safe, sandboxed environment.

Virtual machines have historically been used for server virtualization. virtual machines can perform specific tasks considered too risky to carry out in a host environment, such as accessing virus-infected data or testing operating systems. Since the virtual machine is separated from the rest of the system, the software inside the virtual machine cannot tamper with the host computer.

Types of virtual machines

Users can choose from two different types of virtual machines—process VMs and system VMs:

A **process virtual machine** allows a single process to run as an application on a host machine, providing a platform-independent programming environment by masking the information of the underlying hardware or

operating system. An example of a process VM is the Java Virtual Machine, which enables any operating system to run Java applications as if they were native to that system.

A system virtual machine is fully virtualized to substitute for a physical machine. A system platform supports the sharing of a host computer's physical resources between multiple virtual machines, each running its own copy of the operating system. This virtualization process relies on a hypervisor, which can run on bare hardware, such as VMware ESXi, or on top of an operating system.

(**Hypervisor:** A hypervisor is software to create and fire up virtual machines (VMs). It creates and manages VMs by isolating the operating system and resources from the virtual machines. The hypervisor pools resources like CPU, memory, and storage that can be easily shared between virtual machines and gives each virtual machine the resources that have been allocated. It also manages the scheduling of VM resources against physical resources. Operators control virtual instances of resources such as CPU, memory, storage, and other resources, so guests get the resources on-demand. The physical hardware is still responsible for execution. The CPU is still executing CPU instructions as requested by the VMs, while the hypervisor manages the schedule.)

Virtualization

Virtualization uses software to create an abstraction layer over computer hardware that allows the hardware elements of a single computer—processors, memory, storage and more—to be divided into multiple virtual computers, commonly called virtual machines (VMs). Each VM runs its own operating system (OS) and behaves like an independent computer, even though it is running on just a portion of the actual underlying computer hardware.

It follows that virtualization enables more efficient utilization of physical computer hardware and allows a greater return on an organization's hardware investment.

.Types of virtualization:

All the components of a traditional data center or IT infrastructure can be virtualized today, with various specific types of virtualization:

- **Hardware virtualization:** Hardware virtualization, which is also known as server virtualization, allows hardware resources to be utilized more efficiently and for one machine to simultaneously run different operating systems.

- **Software virtualization:** Software virtualization creates a computer system complete with hardware that allows one or more guest operating systems to run on a physical host machine. For example, Android OS can run on a host machine that is natively using a Microsoft Windows OS, utilizing the same hardware as the host machine does.
- **Storage virtualization:** Storage can be virtualized by consolidating multiple physical storage devices to appear as a single storage device. Benefits include increased performance and speed, load balancing and reduced costs. Storage virtualization also helps with disaster recovery planning, as virtual storage data can be duplicated and quickly transferred to another location, reducing downtime.
- **Network virtualization:** Multiple sub-networks can be created on the same physical network by combining equipment into a single, software-based virtual network resource. Network virtualization also divides available bandwidth into multiple, independent channels, each of which can be assigned to servers and devices in real time. Advantages include increased reliability, network speed, security and better monitoring of data usage.
- **Desktop virtualization:** This common type of virtualization separates the desktop environment from the physical device and stores a desktop on a remote server, allowing users to access their desktops from anywhere on any device. In addition to easy accessibility, benefits of virtual desktops include better data security, cost savings on software licenses and updates, and ease of management.

Advantages of virtual machines

Virtual machines are easy to manage and maintain, and they offer several advantages over physical machines:

- VMs can run multiple operating system environments on a single physical computer, saving physical space, time and management costs.
- Virtual machines support legacy applications, reducing the cost of migrating to a new operating system. For example, a Linux virtual machine running a distribution of Linux as the guest operating system can exist on a host server that is running a non-Linux operating system, such as Windows.
- VMs can also provide integrated disaster recovery and application provisioning options.

Disadvantages of virtual machines

While virtual machines have several advantages over physical machines, there are also some potential disadvantages:

- Running multiple virtual machines on one physical machine can result in unstable performance if infrastructure requirements are not met.
- Virtual machines are less efficient and run slower than a full physical computer. Most enterprises use a combination of physical and virtual infrastructure to balance the corresponding advantages and disadvantages.